

Inigo Quilez :: articles :: fBM - 2019



Intro

fBM stands for Fractional Brownian Motion. But before we talk about nature, fractals and procedural terrains, let's get a bit theoretical for a moment.

A Brownian Motion (BM), without the "fractional" part, is a motion where the position of a given object over time changes in random increments (imagine a sequence of `position+=white_noise();`). Formally, BM is the integral of white noise. These movements define paths that are random yet (statistically) self similar, ie, a zoomed-in version of the path resembles the whole path. A Fractional Brownian Motion is a similar process in which the increments are not completely independent from each other, but there's some sort of memory to the process. If the memory is positively correlated, changes in a given direction will tend to produce future changes in the same direction, and the path will then be smoother than a vanilla BM. If the memory is negatively correlated, a positive change will be most likely followed by a negative change, and the path will be much more random. The parameter that controls the behavior of the memory or the integration, and therefore the self-similarity, its fractal dimension and its power spectrum is called the Hurst Exponent, and it's usually abbreviated as H . Mathematically speaking, H allows us to integrate white noise only partially (say, perform $1/3$ of an integral, hence the "Fractional" part in the name) to design fBMs for any memory characteristics and visual look that we desire. In fact, H takes values between 0 and 1, describing rough and smooth fBMs respectively, where the normal BM happens for $H=1/2$.



fBM() was used to generate the terrain, the clouds, the tree distribution, their color variations, and the canopy details. "Rainforest", 2016: <https://www.shadertoy.com/view/4ttSWf>

Now, that's all very theoretical, and not how us computer graphics folks generate fBM, but I wanted to describe it because it is important to keep its qualities in mind even when doing graphics. Let's see how:

As we know, self-similar structures that are also random are very useful for modeling all sort of natural phenomena procedurally, from **clouds** to **mountains** to **bark textures**. It is intuitively evident that shapes in nature can be decomposed in few big shapes that describe the overall form, and a larger number of medium size shapes that distort the basic contour or surface of the initial shape, and even many more even smaller shapes that add extra detail to the contour and shape of the previous too. This incremental way of adding detail to an object, which allows for an easy way to band limit our shapes for the purposes of LOD (Level Of Detail) and filtering/antialiasing, is an easy one to code and produce visually stunning results. Because of that, it is widely used in films and games. However I believe fBM is not necessarily a well understood mechanism. So this article describes how it functions and their different spectral and visual characteristics for various values of their main parameter H , backed with some experiments and measurements.

Basic Idea

The way fBMs are constructed normally (there are multiple methods) is to invoke deterministic and smooth randomness through some noise function of our choice (**value**, **gradient**, **cellular**, **voronoi**, trigonometric,

simplex, ..., you name it, the choice doesn't matter much), and then construct self-similarity explicitly with it. The fBM does this by starting with a basic noise signal and continuously adding smaller and smaller detailed noise invocations to it. Something like this:

```
float fbm( in vecN x, in float H )
{
    float t = 0.0;
    for( int i=0; i<numOctaves; i++ )
    {
        float f = pow( 2.0, float(i) );
        float a = pow( f, -H );
        t += a*noise(f*x);
    }
    return t;
}
```

This is the purest form of fBM. Each noise() signal (or "wave"), of which we have "numOctaves", gets additively combined with the running total, but it gets compressed horizontally by two effectively reducing its wavelength by two as well, and its amplitude gets reduced exponentially. This accumulation of waves with coordinated reduction of wavelength and amplitude is what produces self-similarity like that seen in nature. After all, in a given space, there's room for just a few big shape changes but there's naturally room for a lot and a lot of tiny ones. Sounds pretty reasonable. In fact, these kind of Power-Law behaviors are found everywhere in nature.

The first thing you might have noticed is that the code above doesn't completely look like most fBM implementations you might have seen in Shadertoy and other code snippets around. This following code is equivalent to the one above, but is far more popular because it avoids the expensive pow() functions:

```
float fbm( in vecN x, in float H )
{
    float G = exp2(-H);
    float f = 1.0;
    float a = 1.0;
    float t = 0.0;
    for( int i=0; i<numOctaves; i++ )
    {
        t += a*noise(f*x);
        f *= 2.0;
        a *= G;
    }
    return t;
}
```

So let's talk about "numOctaves" first. Since each noise is half the wavelength of the previous one (or twice

the frequency), the term for what otherwise should have been "numFrequencies" is replaced by "numOctaves" as a reference to the musical term where a separation of one octave between two notes corresponds to doubling the frequency of the base note. Now, fBMs can be constructed by incrementing the frequency of each noise by something different than two. In that case the term "octave" wouldn't be technically correct anymore, but I've seen people use it regardless. There are cases where you might even want to create waves/noise of frequencies that increase at constant linear rate rather than geometrically, like in an FFT (which can be used indeed to generate periodic fBMs(), which can be useful for ocean textures). But, as we'll later see in this article, for most base noise() functions we can actually increment frequencies in multiples of two, which means we only need a very few iterations, and still get good looking fBMS. In fact, synthesizing the fBM one octave at a time allows us to be very efficient - for example with just 24 octaves/iterations we can create an fBM that covers the whole planet Earth and provides details of just 2 meters. Doing the same with linearly increasing frequencies would take a few orders of magnitude more iterations.

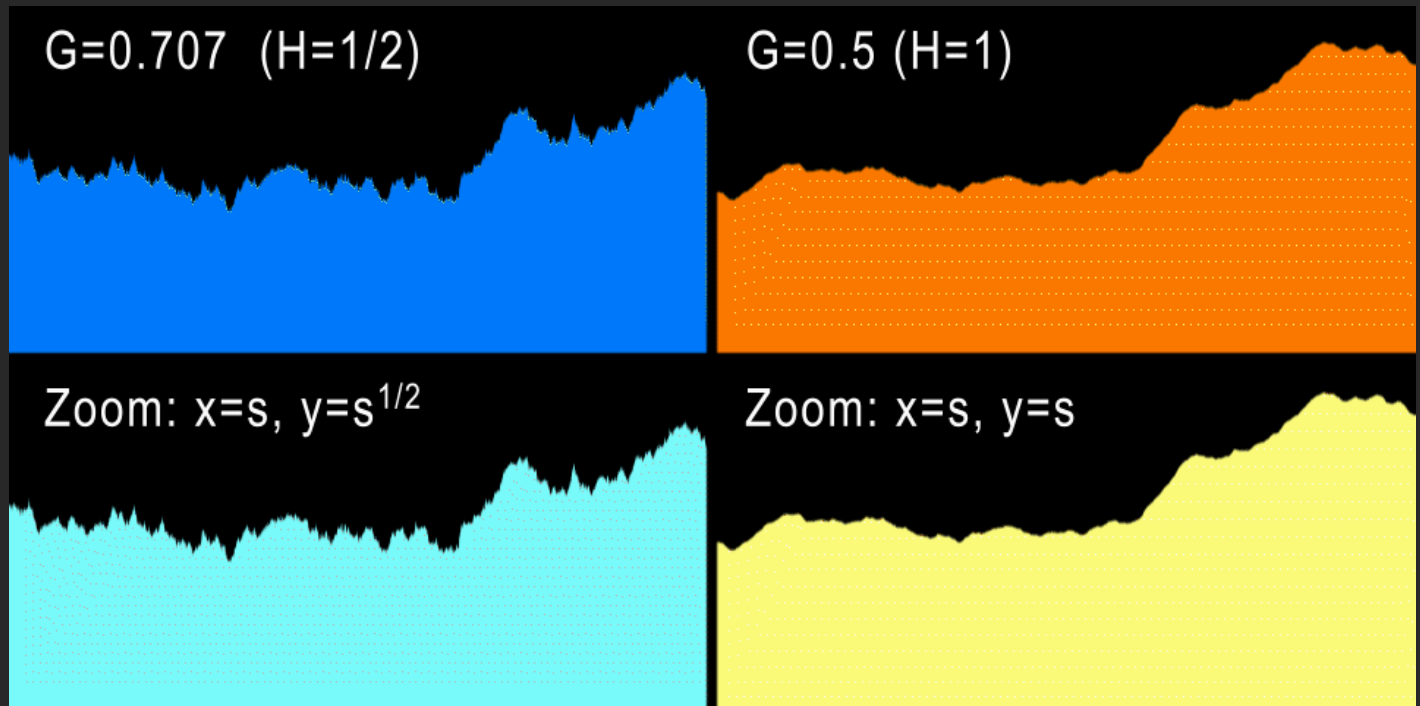
One last note on the sequence of frequencies is that moving from a $f_i = 2^i$ approach to a $f_i = 2 \cdot f_{i-1}$, gives us some flexibility regarding the frequency doubling (or wavelength halving) - we can easily unroll the loop and detune each octave slightly by replacing 2.0 by 2.01, 1.99 and similar values, such that the zeros and peaks of the different noise waves we are accumulating don't superspose exactly, which can sometimes create unrealistic patterns. In the case of 2D fBM one can also rotate the domain a bit besides stretching it by an octave.

Now, in this new code implementation of fBM(), not only we've replaced the frequency generation from a power based formulation to an iterative process, but we've replaced the exponential amplitude decay as well, the Power Law, with a geometric series driven by a "gain" factor G. One needs to convert from H to G by doing $G = 2^{-H}$ which you can derive easily from the first version of the code. However more often than not graphics programmers ignore or don't even know about the Hurst exponent H, and only work with values of G directly. Since we know H goes from 0 to 1, G goes from 1 to 0.5. And indeed, G=0.5 is what most people have hardcoded into their fBM implementations. This hardcoding isn't as flexible as leaving G variable, but there's a good reason to do so, and we are about to see why.

Self Similarity

As we mentioned, the H parameter determines the selfsimilarity of the curve. This is statistical self-similarity of course. So, in the case of a one-dimensional fBM(), if we horizontally zoom-in in it by a factor of U, how much would we need to zoom in vertically in V to get a curve that "looks" the same? Well, since $a = f^{-H}$, then $a \cdot V =$

$(f \cdot U)^{-H} = f^{-H} \cdot U^{-H} = a \cdot U^{-H}$, meaning $V=U^{-H}$. So, if we are zooming in a fBM with a horizontal factor of two, then we'll need to scale vertically with a factor of 2^{-H} . But 2^{-H} is G ! Not coincidentally, when using G to scale our noise amplitudes, we are, by construction, building the self-similarity of fBM with a scale factor of $G = 2^{-H}$.



Left, Brownian Motion ($H=1/2$) and anisotropic zooming. Right fBM ($H=1$) and isotropic zooming.

Code: <https://www.shadertoy.com/view/WsV3zz>

Now, what about our procedural mountains? A naive Brownian Motion has a value of $H=1/2$, which produces a $G=0.707107...$. This generates a curve which looks like itself when zoomed in anisotropically in X and Y (if it's a one-dimensional curve). Indeed, for every horizontal zoom factor U we'd need to scale the curve vertically by $V=\sqrt{U}$, which is not very natural. However, stock market curves do approach $H=1/2$ often, since in theory, each increment or decrement in the value of a stock is independent of its previous changes (remember BM is a process with no memory). In practice of course there are some dependencies and these curves are closer to $H=0.6$.

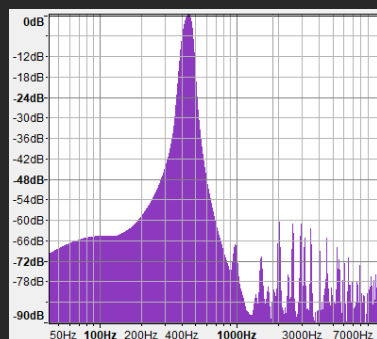
But natural processes have more "memory" into them, and the self-similarity is much more isotropic than that. For example a mountain that is higher is also wider at its base by the same amount, ie, mountains they don't usually stretch or get thinner. So this suggests G should be $1/2$ for mountains - equal zoom in the horizontal and vertical directions. That corresponds to $H=1$, which suggests mountain profiles should be smoother than a stock market curve. And they are, as we'll be measuring actual profiles in a few moments later in this article to confirm this. But we do know from experience that $G=0.5$ produces beautiful fractal terrains and clouds, so $G=0.5$ is indeed the most popular value of G found in all fbm implementations.

But now we have a bit deeper understanding of H , G and fBMs in general. We know that a value of G closer to 1 will make our fBM even wilder than a pure BM, and indeed for $G=1$, which corresponds to $H=0$, we get the noisiest of all the fBMs.

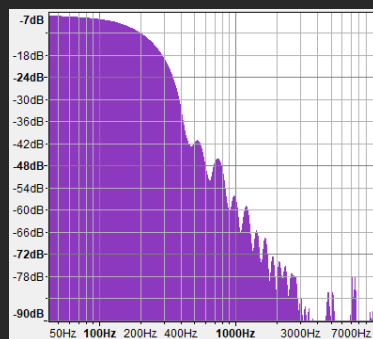
Now, all these parametrized fBMs functions do have names, such as "Pink Noise" for $H=0$, $G=1$ or "Brown Noise" for $H=1/2$, $G=\sqrt{2}$, which are inherited from the field of Digital Signal Processing and well known to people who have sleeping problems. Actually, let's dive a little bit into DSP and compute some spectral characteristics so we gain more intuitions about fBMs.

A Signal Processing look

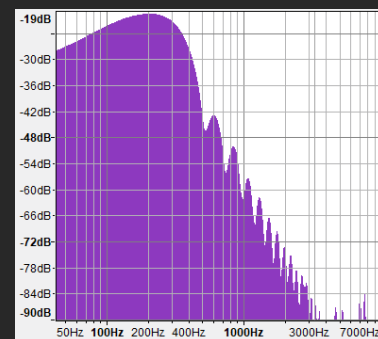
If you think of Fourier analysis, or additive sound synthesis, the fBM() implementation above is similar to that of an Inverse Fourier Transform that is discrete like DFT, although very sparse, and uses a different basic function (basically, it's very different to an IFT, but bear with me). In fact, you can also generate fBM() and CG terrains and even ocean surfaces by performing IFFTs, but it gets very costly quickly. The reason is that IFFT works by additively combining sine waves instead of noise waves, and sine waves are not very efficient at filling the power density spectrum, since each sine wave contributes to a single frequency. However, noise functions have wide spectrums that cover long ranges of frequencies with a single wave. Both **Gradient Noise** and **Value noise** have such rich and thick spectral density plots. Have a look:



Sin Wave



Value Noise

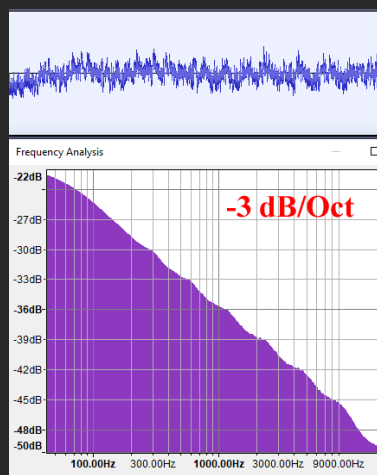


Gradient Noise

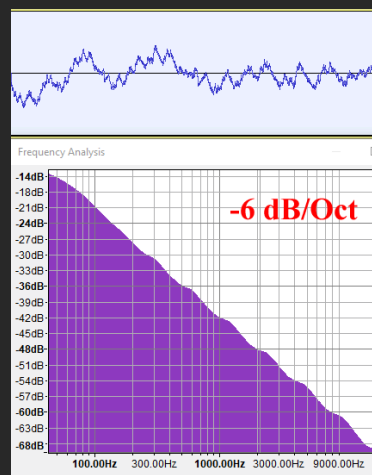
Note how the spectrum of both Value Noise and Gradient Noise have most of the energy concentrated in the low frequencies but are wide - perfect to fill the whole spectrum rapidly with few shifted and rescaled copies. The other problem with sine wave based fBM is that of course it generates repeating patterns which is not

desirable most of the times, although it can become handy for generating tiling textures. The one advantage of $\sin()$ based fBM() is that it is super performant, since trigonometric functions run much faster in hardware than constructing noise with polynomials and hashes/luts, so sometimes it's still **worth using sin based fBM** for performance reasons, even if it produces poor landscapes.

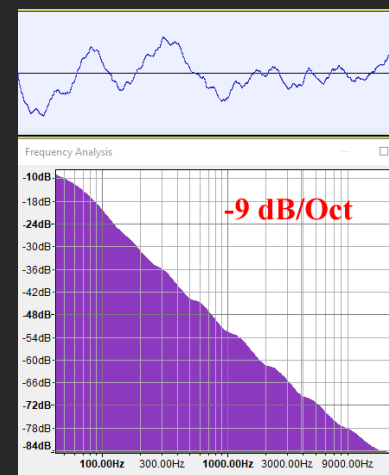
Now let's have a look at the spectral density plots for fBMs of different H values. Pay special attention to the vertical axis labels, since the three graphs are normalized and do not represent the same slopes even if at first glance they all look almost the same. If we call the negative slope of these spectral graphs "B", then, since these graphics are in log-log scale, the spectrum follows a power law of the form f^{-B} . For this test I am using 10 octaves of regular gradient noise to construct these fBMs below.



G=1.0 (H=0)



G=0.707 (H=1/2)



G=0.5 (H=1)

As we can see, the energy of an fBM with $H=0$ ($G=1$) decays at 3db per octave, or basically, inversely to the frequency. This is a power law of f^{-1} ($B=1$), and is called Pink Noise. It sounds like rain.

An fBM() with $H=1/2$ ($G=0.707$) generates a spectrum that decays faster, at 6 db per octave, meaning it has less high frequencies. It does sound indeed deeper, like listening to rain again but from the inside of your room with the window closed. A 6db/Oct decay means the energy is proportional to f^{-2} ($B=2$), and this is indeed the characterization of a Brownian Motion in DSP.

Lastly, our computer graphics favourite fBM, $H=1$ ($G=0.5$), generates a spectral density plot with a 9 db/Octave decay, which means the energy is inversely proportional to the cube of the frequency (f^{-3} , $B=3$). This is an even lower frequency signal, which corresponds with a process with positively correlated memory as we mentioned in the intro. This kind of signal doesn't have a name as far as I know, so I am tempted to call

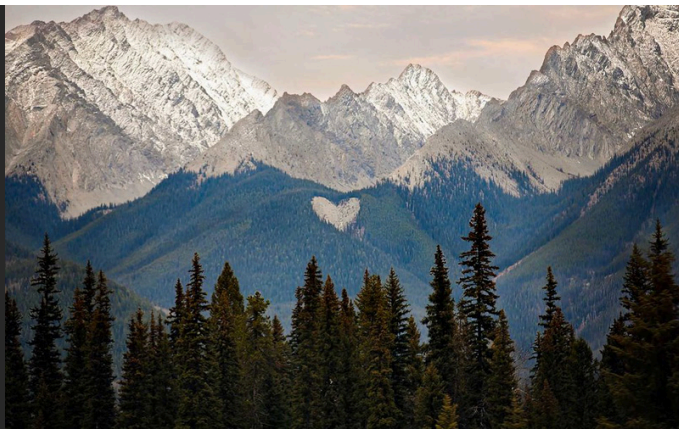
it "Yellow Noise" (just because this color isn't used for some other type of signal). As we know, being isotropic, it models many natural shapes that are self-similar.

Name	H	$G=2^{-H}$	$B=2H+1$	dB/Oct	Sound
Blue	-	-	+1	+3	Spraying Water. Link
White	-	-	0	0	Windy Leaves. Link
Pink	0	1	-1	-3	Rain. Link
Brown	1/2	$\sqrt{2}$	-2	-6	Indoors Rain. Link
Yellow	1	1/2	-3	-9	Engine behind door

Measuring

No, I did some claims about nature being isotropic and therefore being best simulated with yellow noise ($H=1$). So let's put them to some form of testing.

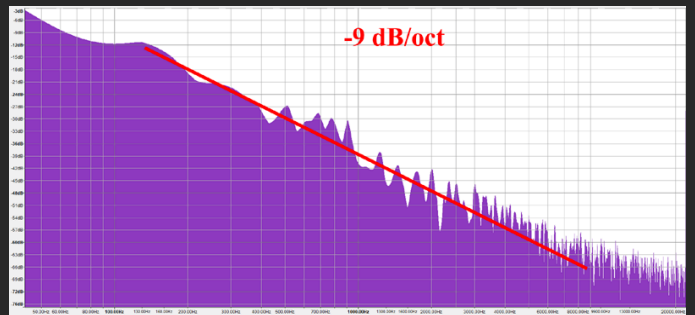
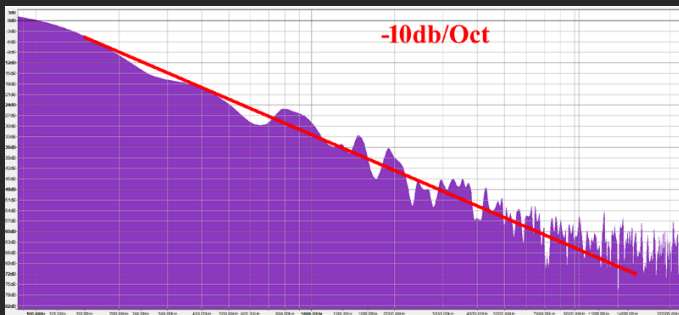
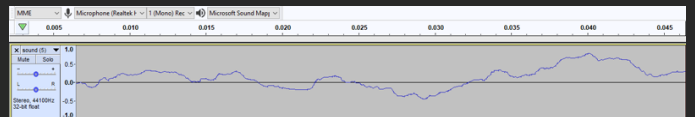
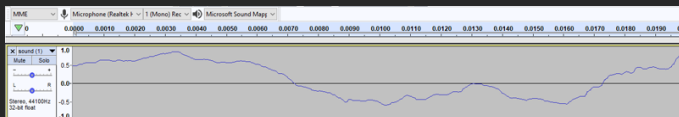
I'd like to caveat that the following is not a rigorous/scientific experiment, but I want to share it here anyways. What I did was to take photos of mountain chains running parallel to the image plane, to prevent perspective distortion. Then I segmented the images in black and white, and converted the sky-mountain interface into a 1D signal. I then interpreted it as a WAV sound file and computed its frequency plot just as with the synthetic fBM() signals I analyzed earlier. I made sure the images were high enough resolution that the FFT algorithm would have something meaningful to work with.



Source: Greek Reporter



Source: Wikipedia



The results seem to indicate that, indeed, mountain profiles follow a -9dB/octave frequency distribution, which corresponds with $B=-3$ or $H=1$ or $G=0.5$, or in other words, Yellow Noise.

While not a rigorous study, this seems to validate our intuition, and also what we already know from experience as computer graphics programmers, namely that $H=1$ ($G=0.5$) produce realistic (isotropic) fractal terrain shapes. But now we just have a better understanding of it, I hope!

inigo quilez - learning computer graphics since 1994